

Unit-7

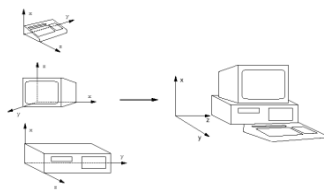
Visible-Surface Detection Methods

More information about Modelling and Perspective Viewing:

Before going to visible surface detection, we first review and discuss the followings:

7.1 Modeling Transformation:

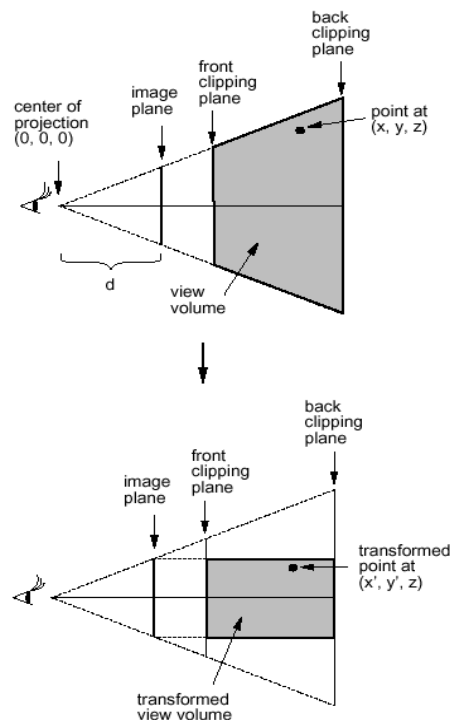
In this stage, we transform objects in their local modelling coordinate



systems into a common coordinate system called the world coordinates.

7.2 Perspective Transformation (in a perspective viewing system):

After Modelling Transformation, Viewing Transformation is carried out to transform objects from the world coordinate system to the viewing



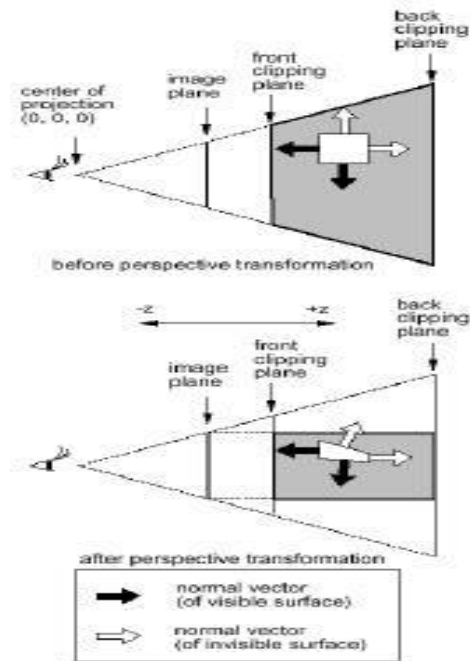
coordinate system. Afterwards, objects in the scene are further perspectively transformed. The effect of such an operation is that after the transformation, the view volume in the shape of a frustum becomes a regular parallelepiped. The transformation equations are shown as follows and are applied to every vertex of each object:

$$x' = x * (d/z),$$

$$y' = y * (d/z),$$

$$z' = z$$

Where (x,y,z) is the original position of a vertex, (x',y',z') is the transformed position of the vertex, and d is the distance of image plane



from the center of projection.

Note that:

Perspective transformation is different from perspective projection:

Perspective projection projects a 3D object onto a 2D plane perspectively.

Perspective transformation converts a 3D object into a deformed 3D object.

After the transformation, the depth value of an object remains unchanged.

Before the perspective transformation, all the projection lines converge to the center of projection. After the transformation, all the projection lines are parallel to each others.

Perspective Projection = Perspective Transformation + Parallel Projection

7.3 Clipping:

In 3D clipping, we remove all objects and parts of objects which are outside of the view volume. Since we have done perspective transformation, the 6 clipping planes,

which form the parallelepiped, are parallel to the 3 axes and hence clipping is straight forward.

Hence the clipping operation can be performed in 2D. For example, we may first perform the clipping operations on the x-y plane and then on the x-z plane.

Problem definition of Visible-Surface Detection Methods:

To identify those parts of a scene that are visible from a chosen viewing position.

Surfaces which are obscured by other opaque surfaces along the line of sight (projection) are invisible to the viewer.

Characteristics of approaches:

- Require large memory size?
- Require long processing time?
- Applicable to which types of objects?

Considerations:

- Complexity of the scene
- Type of objects in the scene
- Available equipment
- Static or animated?

Classification of Visible-Surface Detection Algorithms:

7.4 Object-space Methods

Compare objects and parts of objects to each other within the scene definition to determine which

surfaces, as a whole, we should label as visible:

For each object in the scene do

Begin

1. Determine those part of the object whose view is unobstructed by other parts of it or any other object with respect to the viewing specification.
2. Draw those parts in the object color.

End

- Compare each object with all other objects to determine the visibility of the object parts.
- If there are n objects in the scene, complexity = $O(n^2)$
- Calculations are performed at the resolution in which the objects are defined (only limited by the computation hardware).
- Process is unrelated to display resolution or the individual pixel in the image and the result of the process is applicable to different display resolutions.
- Display is more accurate but computationally more expensive as compared to image space methods because step 1 is typically more complex, eg. Due to the possibility of intersection between surfaces.
- Suitable for scene with small number of objects and objects with simple relationship with each other.

7.5 Image-space Methods (Mostly used)

Visibility is determined point by point at each pixel position on the projection plane.

For each pixel in the image do

Begin

1. Determine the object closest to the viewer that is pierced by the projector through the pixel
2. Draw the pixel in the object colour.

End

- For each pixel, examine all n objects to determine the one closest to the viewer.
- If there are p pixels in the image, complexity depends on n and p ($O(np)$).
- Accuracy of the calculation is bounded by the display resolution.
- A change of display resolution requires re-calculation

Application of Coherence in Visible Surface Detection Methods:

- Making use of the results calculated for one part of the scene or image for other nearby parts.
- Coherence is the result of local similarity
- As objects have continuous spatial extent, object properties vary smoothly within a small local region in the scene. Calculations can then be made incremental.

Types of coherence:

1. Object Coherence:

Visibility of an object can often be decided by examining a circumscribing solid (which may be of simple form, eg. A sphere or a polyhedron.)

2. Face Coherence:

Surface properties computed for one part of a face can be applied to adjacent parts after small incremental modification. (eg. If the face is small, we sometimes can assume if one part of the face is invisible to the viewer, the entire face is also invisible).

3. Edge Coherence:

The Visibility of an edge changes only when it crosses another edge, so if one segment of a nonintersecting edge is visible, the entire edge is also visible.

4. Scan line Coherence:

Line or surface segments visible in one scan line are also likely to be visible in adjacent scan lines.

Consequently, the image of a scan line is similar to the image of adjacent scan lines.

5. Area and Span Coherence:

A group of adjacent pixels in an image is often covered by the same visible object. This coherence is

based on the assumption that a small enough region of pixels will most likely lie within a single polygon. This reduces computation effort in searching for those polygons which contain a given

screen area (region of pixels) as in some subdivision algorithms.

6. Depth Coherence:

The depths of adjacent parts of the same surface are similar.

7. Frame Coherence:

Pictures of the same scene at successive points in time are likely to be similar, despite small changes

in objects and viewpoint, except near the edges of moving objects. Most visible surface detection methods make use of one or more of these coherence properties of a scene. To take advantage of regularities in a scene, eg. Constant relationships often can be established between objects and surfaces in a scene.

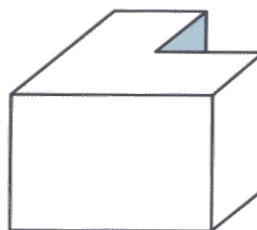
7.6 Back-Face Detection

In a solid object, there are surfaces which are facing the viewer (front faces) and there are surfaces

which are opposite to the viewer (back faces). These back faces contribute to approximately half of the total number of surfaces. Since we cannot see these surfaces anyway, to save processing time, we can remove them before the clipping process with a simple test. Each surface has a normal vector. If this vector is pointing in the direction of the center of projection, it is a front face and can be seen by the viewer. If it is pointing away from the center of projection, it is a back face and cannot be seen by the viewer. The test is very simple, if the z component of the normal vector is positive, then, it is a back face. If the z component of the vector is negative, it is a front face. Note that this technique only caters well for non overlapping convex polyhedral.

For other cases where there are concave polyhedra or

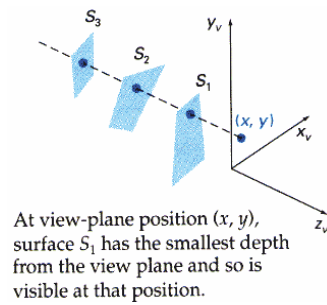
overlapping objects, we still need to apply other methods to further determine where the obscured faces are partially or completely



hidden by other objects (eg. Using Depth-Buffer Method or Depth-sort Method).

7.7 Depth-Buffer Method (Z-Buffer Method)

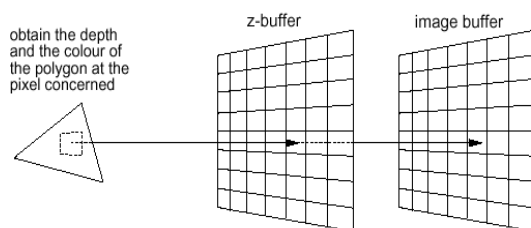
This approach compare surface depths at each pixel



position on the projection plane.

Object depth is usually measured from the view plane

along the z axis of a viewing system. This method requires 2 buffers: one is the image buffer and the other is called the z -buffer (or the depth buffer). Each of these buffers has the same resolution as the image to be



captured. As surfaces are processed, the image buffer is used to store the color values of each pixel position and the z -buffer is used to store the depth values for each (x,y) position.

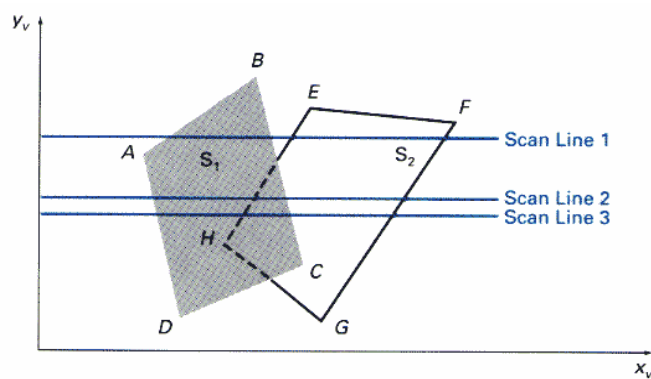
Algorithm:

1. Initially each pixel of the z -buffer is set to the maximum depth value (the depth of the back clipping plane).
 2. The image buffer is set to the background color.
 3. Surfaces are rendered one at a time.
 4. For the first surface, the depth value of each pixel is calculated.
 5. If this depth value is smaller than the corresponding depth value in the z -buffer (ie. it is closer to the view point), both the depth value in the z -buffer and the color value in the image buffer are replaced by the depth value and the color value of this surface calculated at the pixel position.
 6. Repeat step 4 and 5 for the remaining surfaces.
 7. After all the surfaces have been processed, each pixel of the image buffer represents the color of a visible surface at that pixel. This method requires an additional buffer (if compared with the Depth-Sort Method) and the overheads involved in updating the buffer. So this method is less attractive in the cases where only a few objects in the scene are to be rendered.
- Simple and does not require additional data structures.
 - The z -value of a polygon can be calculated incrementally.
 - No pre-sorting of polygons is needed.

- No object-object comparison is required.
- Can be applied to non-polygonal objects.
- Hardware implementations of the algorithm are available in some graphics workstation.
- For large images, the algorithm could be applied to, eg., the 4 quadrants of the image separately, so as to reduce the requirement of a large additional buffer

7.8 Scan-Line Method

In this method, as each scan line is processed, all polygon surfaces intersecting that line are examined to determine which are visible. Across each scan line, depth calculations are made for each overlapping surface to determine which is nearest to the view plane. When the visible surface has been determined, the intensity value for that position is entered into the image buffer.



Scan lines crossing the projection of two surfaces, S_1 and S_2 , in the view plane. Dashed lines indicate the boundaries of hidden surfaces.

For each scan line do

Begin

For each pixel (x,y) along the scan line do ----- Step 1

Begin

$z_buffer(x,y) = 0$

$Image_buffer(x,y) = background_color$

End

For each polygon in the scene do ----- Step 2

Begin

For each pixel (x,y) along the scan line that is covered by the polygon do

Begin

2a. Compute the depth or z of the polygon at pixel location (x,y).

2b. If $z < z_buffer(x,y)$ then

Set $z_buffer(x,y) = z$

Set $Image_buffer(x,y) = polygon's\ colour$

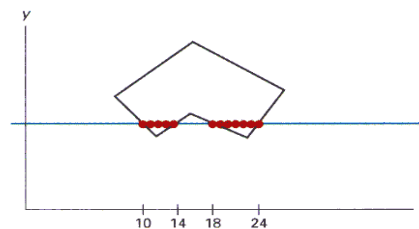
End

End

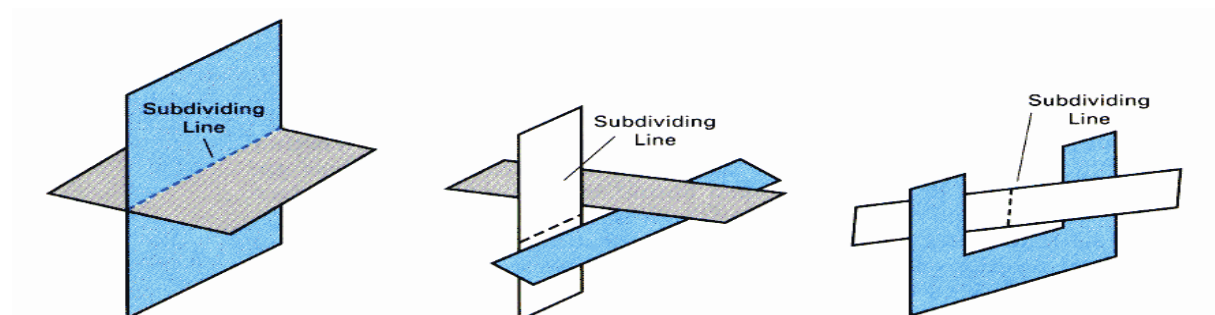
End

- Step 2 is not efficient because not all polygons necessarily intersect with the scan line.
- Depth calculation in 2a is not needed if only 1 polygon in the scene is mapped onto a segment of the scan line.
- To speed up the process:

Recall the basic idea of polygon filling: For each scan line crossing a polygon, this algorithm locates the intersection points of the scan line with the polygon edges. These intersection points are sorted from left to right. Then, we fill the pixels between each intersection pair.



With similar idea, we fill every scan line span by span. When polygon overlaps on a scan line, we perform depth calculations at their edges to determine which polygon should be visible at which span. Any number of overlapping polygon surfaces can be processed with this method. Depth calculations are performed only when there are polygons overlapping. We can take advantage of coherence along the scan lines as we pass from one scan line to the next. If no changes in the pattern of the intersection of polygon edges with the successive scan lines, it is not necessary to do depth calculations. This works only if surfaces do not cut through or otherwise cyclically overlap each other. If cyclic overlap happens, we can divide the surfaces to eliminate the overlaps.



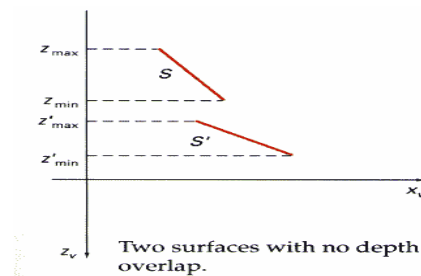
- The algorithm is applicable to non-polygonal surfaces (use of surface and active surface table, zvalue is computed from surface representation).
- Memory requirement is less than that for depth-buffer method.
- Lot of sortings are done on x-y coordinates and on depths.

7.8 Depth-Sort Method

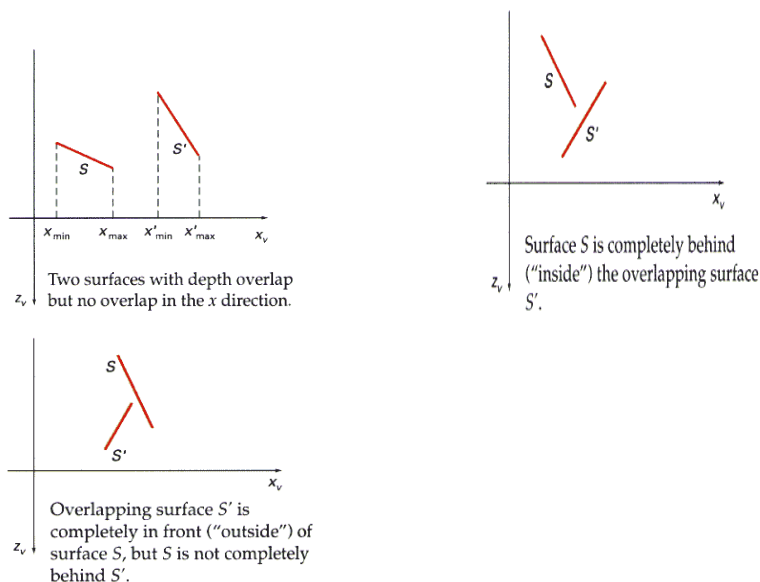
1. Sort all surfaces according to their distances from the view point.
2. Render the surfaces to the image buffer one at a time starting from the farthest surface.
3. Surfaces close to the view point will replace those which are far away.
4. After all surfaces have been processed, the image buffer stores the final image.

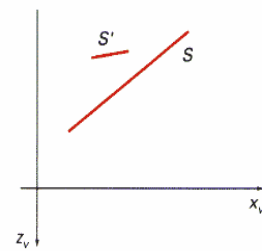
The basic idea of this method is simple. When there are only a few objects in the scene, this method can be very fast. However, as the number of objects increases, the sorting process can become very complex and time consuming.

Example: Assuming we are viewing along the z axis. Surface S with the greatest depth is then compared to other surfaces in the list to determine whether there are any overlaps in depth. If no depth

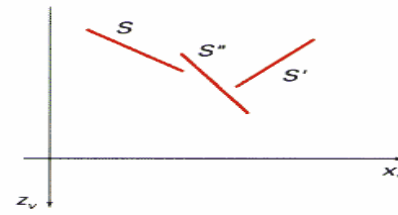


overlaps occur, S can be scan converted. This process is repeated for the next surface in the list. However, if depth overlap is detected, we need to make some additional comparisons to determine whether any of the surfaces should be reordered.





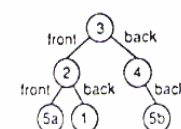
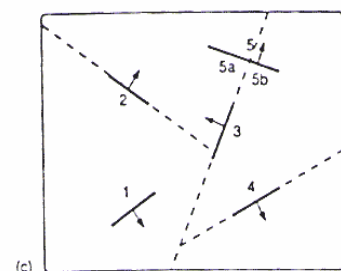
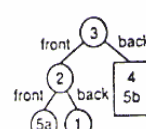
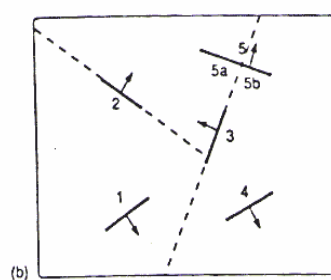
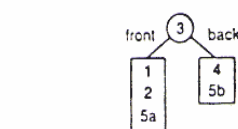
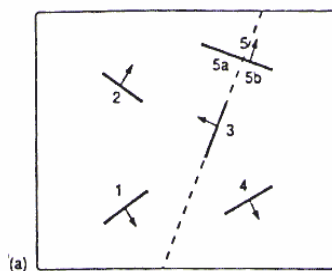
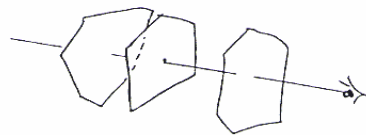
Surface S has greater depth but obscures surface S' .



Three surfaces entered into the sorted surface list in the order S, S', S'' should be reordered S', S'', S .

7.9 Binary Space Partitioning

- suitable for a static group of 3D polygon to be viewed from a number of view points
- based on the observation that hidden surface elimination of a polygon is guaranteed if all polygons on the other side of it as the viewer is painted first, then itself, then all polygons on the same side of it as the viewer



1. The algorithm first build the BSP tree:

- a root polygon is chosen (arbitrarily) which divides the region into 2 half-spaces (2 nodes => front and back)
- a polygon in the front half-space is chosen which divides the half-space into another 2 halfspaces

- the subdivision is repeated until the half-space contains a single polygon (leaf node of the tree)
- the same is done for the back space of the polygon.

2.To display a BSP tree:

- see whether the viewer is in the front or the back

half-space of the root polygon.

- if front half-space then first display back child (subtree) then itself, followed by its front child / subtree
- the algorithm is applied recursively to the BSP tree.

BSP Algorithm

Procedure DisplayBSP(tree: BSP_tree)

Begin

If tree is not empty then

If viewer is in front of the root then

Begin

DisplayBSP(tree.back_child)

displayPolygon(tree.root)

DisplayBSP(tree.front_child)

End

Else

Begin

DisplayBSP(tree.front_child)

displayPolygon(tree.root)

DisplayBSP(tree.back_child)

End

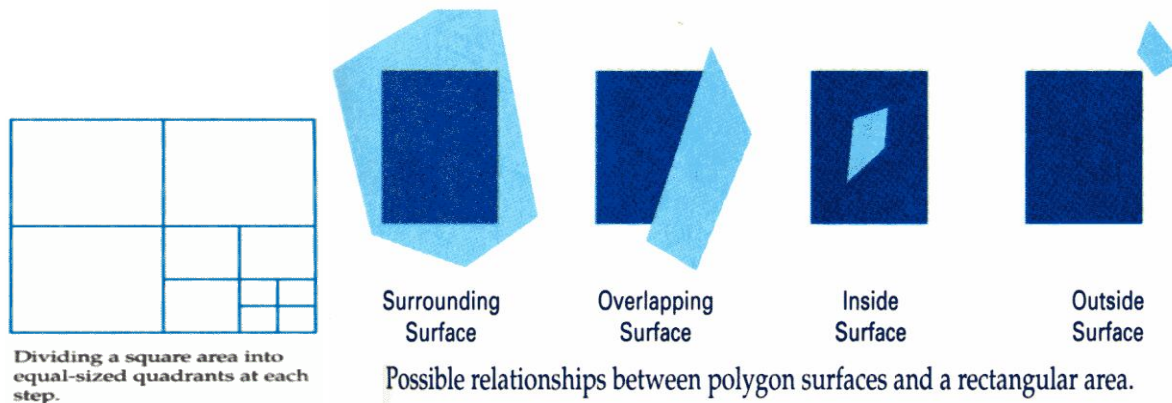
End

Discussion:

- Back face removal is achieved by not displaying a polygon if the viewer is located in its back half-space
- It is an object space algorithm (sorting and intersection calculations are done in object space precision)
- If the view point changes, the BSP needs only minor re-arrangement.
- A new BSP tree is built if the scene changes
- The algorithm displays polygon back to front (cf. Depth-sort)

7.10 Area Subdivision Algorithms

The area-subdivision method takes advantage of area coherence in a scene by locating those view areas that represent part of a single surface. The total viewing area is successively divided into smaller and smaller rectangles until each small area is simple, ie. it is a single pixel, or is covered wholly by a part of a single visible surface or no surface at all.



The procedure to determine whether we should subdivide an area into smaller rectangle is:

1. We first classify each of the surfaces, according to their relations with the area:

Surrounding surface - a single surface completely encloses the area
 Overlapping surface - a single surface that is partly inside and partly outside the area
 Inside surface - a single surface that is completely inside the area
 Outside surface - a single surface that is completely outside the area.

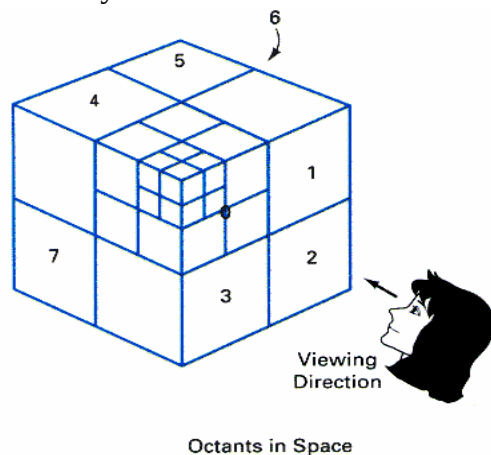
To improve the speed of classification, we can make use of the bounding rectangles of surfaces for early confirmation or rejection that the surfaces should belong to that type.

2. Check the result from 1., that, if any of the following condition is true, then, no subdivision of this area is needed.

- All surfaces are outside the area.
 - Only one surface is inside, overlapping or surrounding surface is in the area.
 - A surrounding surface obscures all other surfaces within the area boundaries.
- For cases b and c, the color of the area can be determined from that single surface.

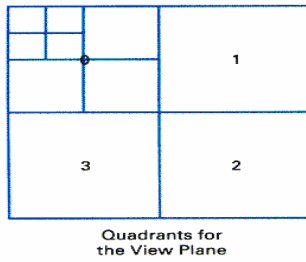
7.11 Octree Methods

In these methods, octree nodes are projected onto the viewing surface in a front-to-back order. Any surfaces toward the rear of the front octants (0,1,2,3) or in the back octants (4,5,6,7) may be hidden by the front surfaces.



With the numbering method (0,1,2,3,4,5,6,7), nodes representing octants 0,1,2,3 for the entire region are visited before the nodes representing octants 4,5,6,7. Similarly the nodes for the front four suboctants of octant 0 are visited before the nodes

for the four back suboctants. When a colour is encountered in an octree node, the corresponding



pixel in the frame buffer is painted only if no previous color has been

loaded into the same pixel position. In most cases, both a front and a back octant must be considered in determining the correct color values for a quadrant. But

- If the front octant is homogeneously filled with some color, we do not process the back octant.
- If the front is empty, it is necessary only to process the rear octant.
- If the front octant has heterogeneous regions, it has to be subdivided and the sub-octants are handled recursively.

Unit-8

Computer Animation

8.1 Overview

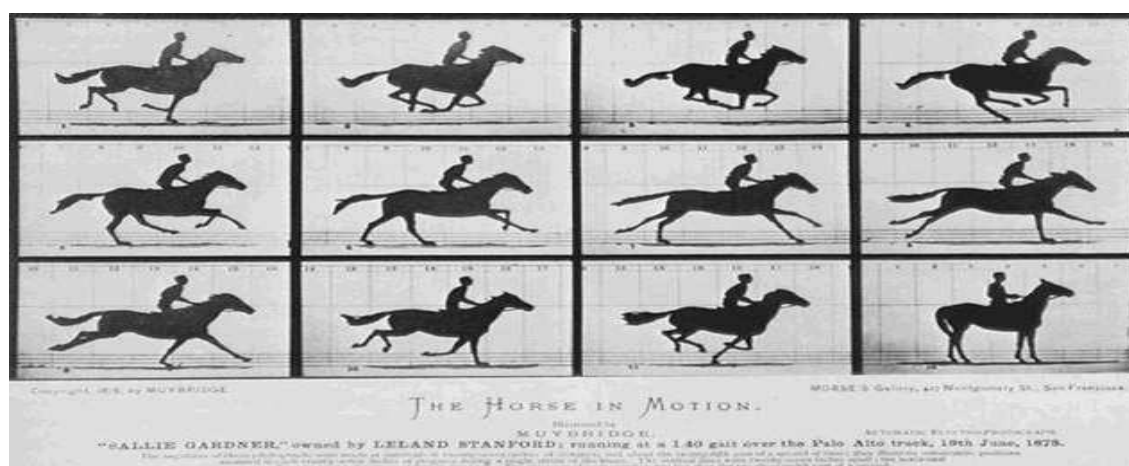
Motion can bring the simplest of characters to life. Even simple polygonal shapes can convey a number of human qualities when animated: identity, character, gender, mood, intention, emotion, and so on. Very simple



Very simple characters (image by Ken Perlin)

A movie is a sequence of frames of still images. For video, the frame rate is typically 24 frames per second. For film, this is 30 frames per second.

Copyright c



In general, animation may be achieved by specifying a model with n parameters that identify degrees of freedom that an animator may be interested in such as

- polygon vertices,
- spline control,
- joint angles,
- muscle contraction,
- camera parameters, or

- color.

With n parameters, this results in a vector $\tilde{q}$ in n -dimensional state space. Parameters may be varied to generate animation. A model's motion is a trajectory through its state space or a set of motion curves for each parameter over time, i.e. $\tilde{q}(t)$, where t is the time of the current frame. Every animation technique reduces to specifying the state space trajectory.

The basic animation algorithm is then: `for t=t1 to tend: render( $\tilde{q}(t)$ ).`

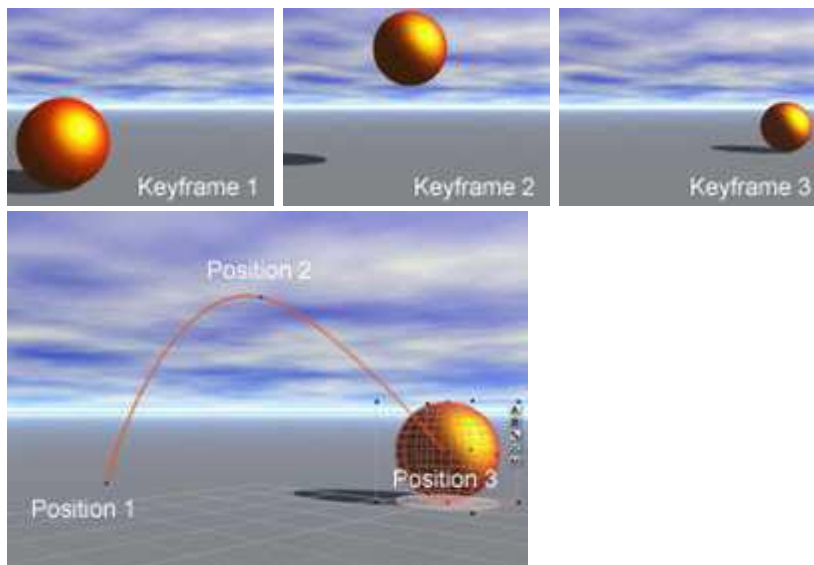
Modeling and animation are loosely coupled. Modeling describes control values and their actions.

Animation describes how to vary the control values. There are a number of animation techniques, including the following:

- User driven animation
 - Keyframing
 - Motion capture
- Procedural animation
 - Physical simulation
 - Particle systems
 - Crowd behaviors
- Data-driven animation

8.2 Keyframing

Keyframing is an animation technique where motion curves are interpolated through states at times, $(\tilde{q}_1, \dots, \tilde{q}_T)$, called keyframes, specified by a user



Catmull-Rom splines are well suited for keyframe animation because they pass through their control points.

- Pros:
 - Very expressive

- Animator has complete control over all motion parameters
- Cons:

Very labor intensive

- Difficult to create convincing physical realism

• Uses:

- Potentially everything except complex physical phenomena such as smoke, water, or fire

8.3 Kinematics

Kinematics describe the properties of shape and motion independent of physical forces that cause motion. Kinematic techniques are used often in keyframing, with an animator either setting joint parameters explicitly with **forward kinematics** or specifying a few key joint orientations and having the rest computed automatically with **inverse kinematics**.

16.3.1 Forward Kinematics

With forward kinematics, a point $\bar{p}$ is positioned by $\bar{p} = f(\underline{\theta})$ where $\underline{\theta}$ is a state vector $(\theta_1, \theta_2, \dots, \theta_n)$ specifying the position, orientation, and rotation of all joints.

For the above example, $\bar{p} = (l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2), l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2))$.

Inverse Kinematics

With inverse kinematics, a user specifies the position of the end effector, $\bar{p}$, and the algorithm has to evaluate the required $\underline{\theta}$ give $\bar{p}$. That is, $\underline{\theta} = f^{-1}(\bar{p})$.

Usually, numerical methods are used to solve this problem, as it is often nonlinear and either underdetermined or overdetermined. A system is underdetermined when there is not a unique solution, such as when there are more equations than unknowns. A system is overdetermined when it is inconsistent and has no solutions.

Extra constraints are necessary to obtain unique and stable solutions. For example, constraints may be placed on the range of joint motion and the solution may be required to minimize the kinetic energy of the system.

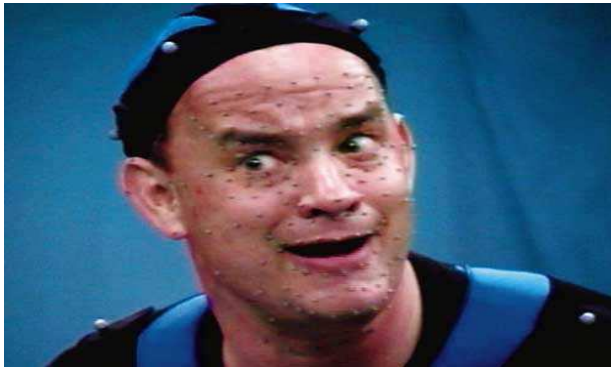
8.3.1 Motion Capture

In motion capture, an actor has a number of small, round markers attached to his or her body that reflect light in frequency ranges that motion capture cameras are specifically designed to pick up



image from movement.nyu.edu)

With enough cameras, it is possible to reconstruct the position of the markers accurately in 3D. In practice, this is a laborious process. Markers tend to be hidden from cameras and 3D reconstructions fail, requiring a user to manually fix such drop outs. The resulting motion curves are often noisy, requiring yet more effort to clean up the motion data to more accurately match what an animator wants. Despite the labor involved, motion capture has become a popular technique in the movie and game industries, as it allows fairly accurate animations to be created from the motion of actors. However, this is limited by the density of markers that can be placed on a single actor. Faces, for example, are still very difficult to convincingly reconstruct.



Pros:

- Captures specific style of real actors

• Cons:

- Often not expressive enough
- Time consuming and expensive
- Difficult to edit

• Uses:

- Character animation
- Medicine, such as kinesiology and biomechanics

